

SmartAI ChatBot - Complete Product Requirements Document (PRD)

Version: 2.0 | **Status:** Production Ready | **Last Updated:** 2026
Owner: Product & Engineering Team | **Stakeholders:** All Teams

Table of Contents

1. [Executive Summary](#)
 2. [Product Overview](#)
 3. [Target Users & Personas](#)
 4. [Core Features & Modules](#)
 5. [Technical Architecture](#)
 6. [Database Schema](#)
 7. [API Specifications](#)
 8. [Security Requirements](#)
 9. [Deployment & DevOps](#)
 10. [Development Roadmap](#)
 11. [Acceptance Criteria](#)
-

1. Executive Summary

SmartAI ChatBot is an intelligent, multi-modal AI platform that enables users to:

- Chat with multiple AI models (OpenAI, Google Gemini, Anthropic Claude, Groq, Ollama)
- Upload and analyze PDFs with intelligent Q&A
- Process images with OCR and vision AI
- Interact via voice (speech-to-text and text-to-speech)

- Maintain persistent chat history
- Access admin analytics dashboard

Primary Goal: Build a scalable, enterprise-grade SaaS platform that supports individual users, businesses, and eventually becomes a subscription-based revenue model.

Go-to-Market: Launch with free tier (50 messages/day), then introduce Pro/Enterprise tiers.

2. Product Overview

2.1 Product Name & Vision

- **Name:** SmartAI ChatBot
- **Vision:** "Empower everyone to leverage AI for work, learning, and creativity"
- **Mission:** Provide an accessible, feature-rich AI platform with minimal friction

2.2 Key Differentiators

Feature	SmartAI	Competitors
Multi-AI Model Support	✔️ (5+ Models)	Limited
PDF + Image + Voice Support	✔️ Full	Partial
Real-time Streaming	✔️ WebSocket	❌
Chat Export/Share	✔️ Multiple Formats	Limited
Admin Dashboard	✔️ Full Analytics	❌
Self-hosted Option (Ollama)	✔️	❌

3. Target Users & Personas

3.1 Primary Users

1. **Students** - Learning, assignment help, research
2. **Developers** - Code generation, debugging, learning
3. **Teachers** - Quiz generation, content creation
4. **Freelancers** - Content writing, translation, coding projects
5. **Businesses** - Customer support, content creation, data analysis
6. **Enterprises** - Custom AI workflows, analytics, compliance

3.2 User Personas

Persona 1: Ravi (Student)

- Age: 20-25
- Pain Point: Needs help understanding concepts, generating notes
- Usage: Daily, 20-30 messages/day
- Device: Mobile + Laptop

Persona 2: Priya (Software Developer)

- Age: 25-35
- Pain Point: Code debugging, learning new languages
- Usage: 50+ messages/day, PDF uploads for documentation
- Device: Laptop, Occasionally Mobile

Persona 3: Arjun (Entrepreneur)

- Age: 30-45
- Pain Point: Content creation, business strategy, automation
- Usage: 100+ messages/day, Image & PDF analysis
- Device: Mobile-first, Website secondary

- Subscription: Likely to upgrade to Pro
-

4. Core Features & Modules

4.1 Authentication Module

Status:  Critical | **Priority:** P0

Features:

-  Email/Password Registration with validation
-  Login with JWT Authentication
-  Refresh Token mechanism (15 min access, 7 day refresh)
-  Google OAuth 2.0 Integration
-  GitHub OAuth Integration
-  Forgot Password with email verification
-  Email verification token (24hr expiry)
-  Two-factor authentication (Phase 3)

Technical Details:

JWT Structure:

- Header: {alg: "HS256", typ: "JWT"}
- Payload: {user_id, email, role, exp, iat}
- Signature: HMAC-SHA256

Password Requirements:

- Min 8 characters
- 1 uppercase, 1 lowercase, 1 number, 1 special char

API Endpoints:

- `POST /api/auth/register` - Register new user

- `POST /api/auth/login` - Login user
 - `POST /api/auth/logout` - Logout (blacklist token)
 - `POST /api/auth/refresh` - Refresh JWT
 - `POST /api/auth/google` - Google OAuth callback
 - `POST /api/auth/github` - GitHub OAuth callback
 - `POST /api/auth/forgot-password` - Send reset email
 - `POST /api/auth/reset-password` - Reset password with token
 - `POST /api/auth/verify-email` - Verify email token
-

4.2 User Dashboard Module

Status:  Critical | **Priority:** P0

Features:

-  Home Dashboard with quick stats
-  Recent Chats (last 10 conversations)
-  Pinned Chats (user favorites)
-  User Profile Management
-  Subscription Status Display
-  Settings (Dark Mode, Notifications, Privacy)
-  Notification Center
-  User Activity Log

Dashboard Widgets:

Quick Stats:

- Total Messages This Week
- PDFs Analyzed
- Images Processed
- AI Models Used

Recommendations:

- Trending Prompts
- Suggested Features

API Endpoints:

- `GET /api/user/profile` - Get user details
 - `PUT /api/user/profile` - Update profile (name, avatar, bio)
 - `GET /api/user/stats` - Get user statistics
 - `POST /api/user/settings` - Update settings
 - `GET /api/user/notifications` - Get notifications
 - `PUT /api/user/notifications/read` - Mark notifications as read
 - `DELETE /api/user/account` - Delete account permanently
-

4.3 Chat Module (Core)

Status:  Critical | **Priority:** P0

Features:

-  Create New Chat
-  Rename Chat
-  Delete Chat with confirmation
-  Pin/Unpin important chats

- ☒ Search across all chats
- ☒ Export chat (JSON, PDF, Markdown, DOCX)
- ☒ Share chat (public link, read-only)
- ☒ Continue conversation (reload context)
- ☒ Typing indicator
- ☒ Real-time message streaming
- ☒ Message editing (within 5 min)
- ☒ Message deletion
- ☒ Copy message to clipboard

Chat Management:

Chat States:

- active: Currently in use
- archived: User archived but kept
- deleted: Soft delete (30-day recovery)
- shared: Publicly shared chat

Message Roles:

- user: User message
- assistant: AI response
- system: System notifications

API Endpoints:

- `GET /api/chats` - List all chats with pagination
- `POST /api/chats` - Create new chat
- `GET /api/chats/{chat_id}` - Get specific chat
- `PUT /api/chats/{chat_id}` - Update chat (title, pin status)
- `DELETE /api/chats/{chat_id}` - Delete chat (soft delete)

- `POST /api/chats/{chat_id}/export` - Export chat
 - `POST /api/chats/{chat_id}/share` - Generate share link
 - `GET /api/chats/{chat_id}/messages` - Get chat messages
 - `POST /api/chats/{chat_id}/messages` - Add message (streaming)
 - `PUT /api/messages/{message_id}` - Edit message
 - `DELETE /api/messages/{message_id}` - Delete message
 - `POST /api/chats/search` - Full-text search
-

4.4 AI Chat Features

Status:  Critical | **Priority:** P0

4.4.1 Text Chat

Features:

- Ask any question to AI
- Multi-turn conversations with context
- Adjustable response length (short/medium/long)
- Temperature control (0-1 scale for creativity)
- Stop sequences customization
- Model selection per message

Supported AI Models:

Provider: OpenAI

- GPT-4 (Premium)
- GPT-3.5 Turbo (Default)
- GPT-4 Vision (Image support)

Provider: Google

- Gemini Pro (v1.5)
- Gemini Vision

Provider: Anthropic

- Claude 3 Opus (Premium)
- Claude 3 Sonnet
- Claude 3 Haiku (Fast, Free tier)

Provider: Groq

- Mixtral 8x7B
- Llama 70B
- (Ultra-fast inference)

Provider: Ollama

- Local deployment
- Custom models
- Privacy-focused

4.4.2 Code Generation & Debugging

Features:

- Generate code in multiple languages
- Explain code functionality
- Find and fix errors
- Code optimization suggestions

- Language support:
 - Python, JavaScript/Node.js, Java, C++, C#, Go, Rust, PHP, SQL

Endpoints:

- `POST /api/ai/generate-code` - Generate code
- `POST /api/ai/debug-code` - Debug code
- `POST /api/ai/explain-code` - Explain code
- `POST /api/ai/optimize-code` - Optimize code

4.4.3 Translator

Features:

- Translate between 50+ languages
- Maintain tone and context
- Batch translation support
- Slang and idiom handling

Supported Languages:

- English, Hindi, Spanish, French, German, Chinese (Simplified & Traditional), Japanese, Arabic, Portuguese, Russian, Italian, Korean, Turkish, Polish, Dutch, Swedish, Greek, Hebrew, Thai, Vietnamese, Indonesian, Tagalog, Bengali, Urdu, and 25+ more

Endpoint:

- `POST /api/ai/translate` - Translate text

4.4.4 Summarizer

Features:

- Summarize articles (configurable length: 25%, 50%, 75%)

- Summarize research papers (with abstract extraction)
- Summarize PDFs
- Extract key points
- Generate bullet-point summary
- Extractive + Abstractive summarization

Endpoint:

- `POST /api/ai/summarize` - Summarize content

4.4.5 Grammar & Writing Enhancement

Features:

- Correct grammar errors with explanations
- Improve writing clarity
- Professional tone conversion
- Casual to formal conversion
- Expand text with examples
- Tone detection

Endpoint:

- `POST /api/ai/grammar-fix` - Grammar & writing assistance

4.4.6 AI Teacher

Features:

- Explain complex concepts in simple terms
- Generate practice quizzes (multiple choice, short answer)
- Create assignments with solutions
- Provide real-world examples
- Generate interview questions for interview prep

- Difficulty level adjustment (beginner, intermediate, advanced)

Endpoint:

- `POST /api/ai/teacher` - Educational content generation
-

4.5 PDF Chat Module

Status:  High Priority | **Priority:** P1

Workflow:

```
User uploads PDF
  ↓
Backend validates file (max 50MB, PDF format)
  ↓
Extract text using PyMuPDF
  ↓
Split into chunks (500 chars overlap: 50 chars)
  ↓
Generate embeddings (Sentence Transformers)
  ↓
Store in FAISS vector database
  ↓
User asks questions
  ↓
Retrieve relevant chunks (top-5 by similarity)
  ↓
Augment prompt with context
  ↓
Send to AI model
  ↓
Stream response back to user
```

Technical Stack:

- **PDF Extraction:** PyMuPDF (fitz)
- **Text Processing:** LangChain
- **Embeddings:** Sentence-Transformers (all-MiniLM-L6-v2)
- **Vector DB:** FAISS (with persistent storage)
- **Chunking Strategy:** Recursive character split

Features:

- ☒ Upload single/multiple PDFs
- ☒ Automatic text extraction
- ☒ Smart chunking with overlap
- ☒ Semantic search on PDF content
- ☒ Q&A with context awareness
- ☒ Highlight relevant passages
- ☒ Download references
- ☒ Support for scanned PDFs (OCR integration)
- ☒ Multi-language PDF support

Storage:

- PDFs stored in AWS S3/Cloudinary
- FAISS index stored in PostgreSQL (serialized)
- Metadata in relational DB

API Endpoints:

- `POST /api/pdf/upload` - Upload PDF
- `GET /api/pdf/files` - List user PDFs
- `POST /api/pdf/chat` - Ask question on PDF

- `GET /api/pdf/{pdf_id}/summary` - Auto-generate summary
 - `DELETE /api/pdf/{pdf_id}` - Delete PDF
 - `POST /api/pdf/batch-upload` - Upload multiple PDFs
-

4.6 Image Chat Module

Status: ✔ High Priority | **Priority:** P1

Workflow:

```
graph TD; A[User uploads image] --> B[Validate image (max 20MB, JPG/PNG/WebP)]; B --> C[Extract text via OCR (Tesseract)]; C --> D[Extract visual features (OpenCV)]; D --> E[Send to Vision AI (Gemini Vision or GPT-4V)]; E --> F[Analyze and describe]; F --> G[User asks questions]; G --> H[AI answers based on image context];
```

Technical Stack:

- **Image Processing:** OpenCV, Pillow
- **OCR:** Tesseract
- **Vision AI:** Google Gemini Vision, OpenAI GPT-4V
- **Image Storage:** AWS S3/Cloudinary

Features:

- ☒ Upload JPG, PNG, WebP, GIF
- ☒ Automatic OCR text extraction
- ☒ Visual description generation
- ☒ Object detection and labeling
- ☒ Q&A on image content
- ☒ Text extraction (scanned documents)
- ☒ Diagram analysis
- ☒ Chart interpretation
- ☒ Logo/brand identification
- ☒ NSFW/Safety filtering

API Endpoints:

- `POST /api/image/upload` - Upload image
 - `GET /api/image/files` - List user images
 - `POST /api/image/analyze` - Analyze image
 - `POST /api/image/chat` - Ask question on image
 - `POST /api/image/ocr` - Extract text from image
 - `DELETE /api/image/{image_id}` - Delete image
-

4.7 Voice Chat Module

Status: ☒ High Priority | **Priority:** P1

Speech-to-Text Workflow:

User records/uploads audio



Validate audio format (max 25MB, MP3/WAV/M4A)



Send to Whisper API



Get transcribed text



Process as regular chat message

Text-to-Speech Workflow:

AI generates response text



Send to TTS service (gTTS/Edge TTS)



Generate MP3 audio



Stream audio to frontend



User can download or listen

Technical Stack:

- **Speech-to-Text:** OpenAI Whisper API
- **Text-to-Speech:** Google gTTS, Microsoft Edge TTS
- **Audio Storage:** AWS S3/Cloudinary
- **Audio Formats:** MP3, WAV, M4A, OGG

Features:

- ☒ Record audio directly in browser
- ☒ Upload pre-recorded audio
- ☒ Real-time transcription
- ☒ Support multiple languages
- ☒ Noise filtering
- ☒ Voice response in selected language
- ☒ Speaker selection (male/female/neutral)
- ☒ Speech rate adjustment (0.5x - 2x)
- ☒ Audio quality selection

API Endpoints:

- `POST /api/voice/transcribe` - Convert audio to text
 - `POST /api/voice/synthesize` - Convert text to audio
 - `POST /api/voice/record` - WebSocket for real-time transcription
 - `DELETE /api/voice/{audio_id}` - Delete audio file
-

4.8 Chat History & Context Management

Status: ☒ High Priority | **Priority:** P1

Storage Structure:

Chat

```
├─ id
├─ user_id
├─ title
├─ created_at
├─ updated_at
├─ ai_model_used
└─ is_pinned
    │
    │   ── Messages
    │   │   ├── id
    │   │   ├── chat_id
    │   │   ├── role (user/assistant/system)
    │   │   ├── content
    │   │   ├── tokens_used
    │   │   ├── ai_model
    │   │   ├── timestamp
    │   │   └─ embeddings (for search)
    │
    └─ Attachments
        ├── id
        ├── type (pdf/image/audio)
        ├── file_url
        ├── file_name
        └─ upload_timestamp
```

Features:

-  Auto-save chat history
-  Store AI model used per message
-  Token usage tracking

- ☒ Full-text search on messages
- ☒ Semantic search using embeddings
- ☒ Filter by date, AI model, type
- ☒ Quick access to recent chats
- ☒ Context window management (last 10-50 messages)
- ☒ Conversation continuity across sessions

Context Management:

For each chat request:

1. Retrieve last N messages (configurable: 5-50)
2. Include system prompt
3. Include user message
4. Add PDF/Image context if uploaded
5. Calculate total tokens
6. Truncate if exceeds model limit
7. Send to AI model

API Endpoints:

- `GET /api/messages/{chat_id}` - Get chat messages
 - `POST /api/messages/search` - Search messages
 - `GET /api/messages/semantic-search` - Semantic search
 - `DELETE /api/messages/{message_id}` - Delete message
-

4.9 Admin Dashboard Module

Status: ☒ High Priority | **Priority:** P1

Access: Only users with `admin` role

Dashboard Sections:

4.9.1 Overview

Key Metrics:

- Total Active Users (daily, weekly, monthly)
- Premium Users Count
- Messages Generated Today
- AI Models Usage Distribution
- API Response Times
- Error Rate (%)

4.9.2 Users Management

Displays:

- User list with pagination
- Search by name/email/ID
- User status (active/inactive/suspended)
- Subscription status
- Last login date
- Total messages sent

Actions:

- View user details
- Suspend/unsuspend user
- Reset user password
- Export user data
- Send announcement

4.9.3 Analytics & Reporting

Charts & Graphs:

- Daily active users trend
- AI model usage distribution
- Revenue tracking (if subscriptions)
- Feature adoption rates
- Chat count by feature
- Error logs with stack traces

Exportable Reports:

- Daily/Weekly/Monthly summaries
- User engagement reports
- Revenue reports
- API usage reports
- Error reports

4.9.4 Server Status & Health

Monitoring:

- API response times
- Database connection health
- Redis cache status
- AWS S3 connectivity
- AI API integrations status
- Error rates by endpoint
- Uptime percentage

Alerts:

- High error rate (>5%)
- Slow API responses (>2s)
- Database connection failures
- Storage quota alerts

4.9.5 API Usage & Costs

Tracking:

- API calls per user
- Tokens consumed
- Cost per AI model
- Peak usage times
- Cost breakdown by feature
- Budget warnings

API Cost Calculation:

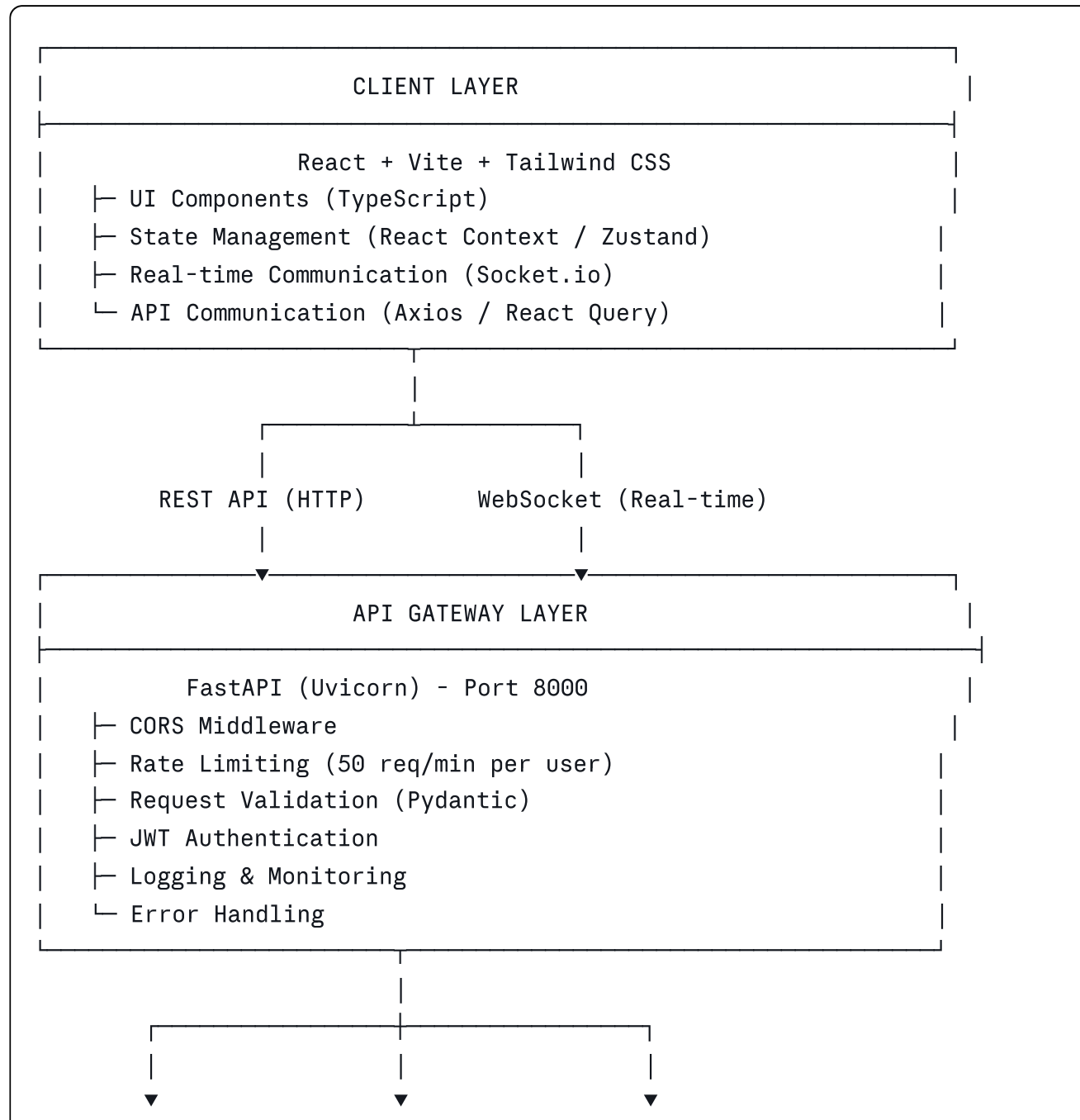
$\text{Cost} = (\text{tokens_used} * \text{rate_per_1k_tokens}) + \text{file_processing_cost}$

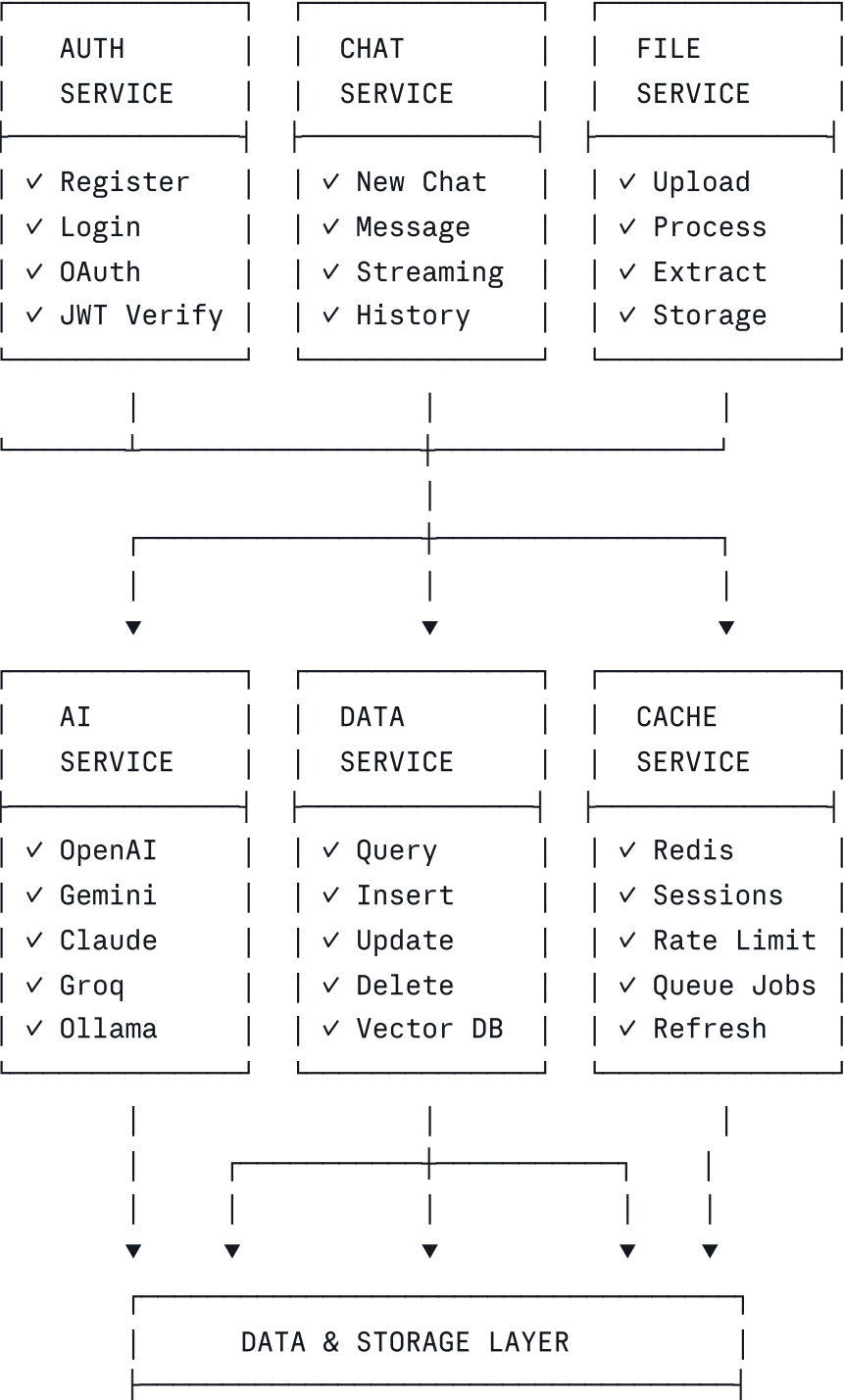
API Endpoints (Admin only):

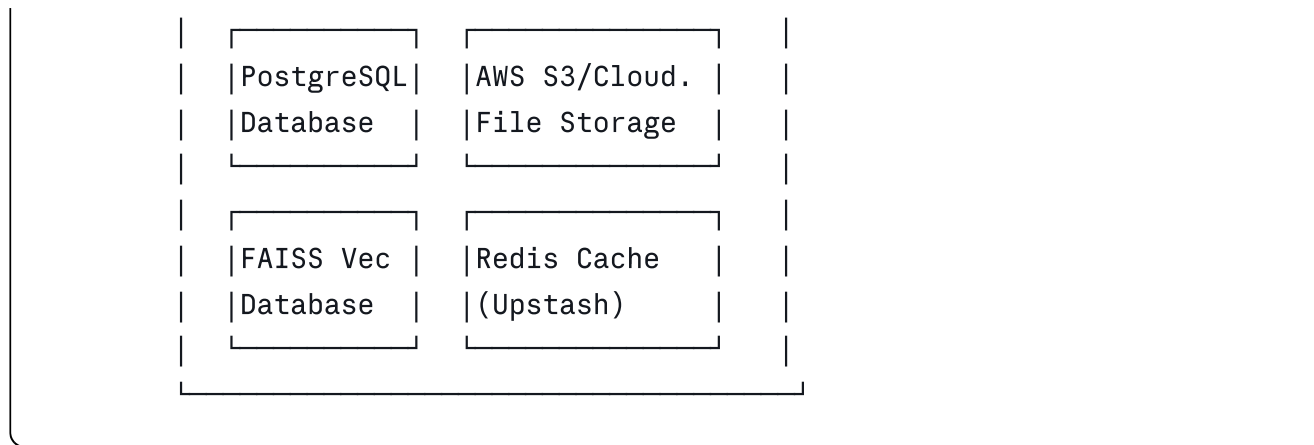
- `GET /api/admin/dashboard` - Dashboard overview
 - `GET /api/admin/users` - Users list with filtering
 - `PUT /api/admin/users/{user_id}/status` - Change user status
 - `GET /api/admin/analytics` - Detailed analytics
 - `GET /api/admin/server-status` - Server health
 - `GET /api/admin/api-logs` - API usage logs
 - `GET /api/admin/reports/export` - Export reports
 - `POST /api/admin/announcements` - Send announcement
 - `DELETE /api/admin/users/{user_id}` - Delete user
-

5. Technical Architecture

5.1 Architecture Diagram







5.2 Technology Stack Details

Frontend:

- React 18+ with TypeScript
- Vite (build tool, ~300ms cold start)
- Tailwind CSS + Shadcn UI
- React Router v6 (routing)
- Axios + React Query (data fetching)
- Zustand or Context API (state management)
- Socket.io-client (WebSocket)
- React Hook Form + Zod (form validation)
- Framer Motion (animations)

Backend:

- Python 3.11+
- FastAPI (async framework)
- Uvicorn (ASGI server)
- SQLAlchemy 2.0 (ORM)
- Alembic (migrations)

- Pydantic (validation)
- JWT-Python (authentication)
- Passlib + bcrypt (password hashing)
- WebSockets (real-time)
- Celery (task queue)

AI & ML:

- OpenAI API (GPT-4, GPT-3.5)
- Google Gemini API
- Anthropic Claude API
- Groq API
- Ollama (self-hosted)
- LangChain (LLM framework)
- Sentence-Transformers (embeddings)
- FAISS (vector database)
- PyMuPDF (PDF extraction)
- OpenCV (image processing)
- Tesseract (OCR)
- Whisper (speech-to-text)
- gTTS/Edge TTS (text-to-speech)

Database & Cache:

- PostgreSQL 15+ (relational DB)
- Redis/Upstash (cache & queue)
- FAISS (vector database)

File Storage:

- AWS S3 or Cloudinary

Deployment:

- Frontend: Vercel
 - Backend: Railway/Render/AWS EC2
 - Database: Supabase PostgreSQL
 - Redis: Upstash
 - Monitoring: Sentry, New Relic
-

6. Database Schema

6.1 Core Tables

Users Table

```
sql
```

```
CREATE TABLE users (  
  id UUID PRIMARY KEY DEFAULT gen_random_uuid(),  
  email VARCHAR(255) UNIQUE NOT NULL,  
  username VARCHAR(100) UNIQUE NOT NULL,  
  password_hash VARCHAR(255) NOT NULL,  
  full_name VARCHAR(255),  
  avatar_url VARCHAR(500),  
  bio TEXT,  
  role ENUM('user', 'admin', 'moderator') DEFAULT 'user',  
  google_id VARCHAR(255) UNIQUE,  
  github_id VARCHAR(255) UNIQUE,  
  is_active BOOLEAN DEFAULT TRUE,  
  is_email_verified BOOLEAN DEFAULT FALSE,  
  created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,  
  updated_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,  
  last_login TIMESTAMP,  
  deleted_at TIMESTAMP, -- Soft delete  
  INDEX(email),  
  INDEX(google_id),  
  INDEX(github_id)  
);
```

Chats Table

sql

```
CREATE TABLE chats (  
  id UUID PRIMARY KEY DEFAULT gen_random_uuid(),  
  user_id UUID NOT NULL REFERENCES users(id) ON DELETE CASCADE,  
  title VARCHAR(255) NOT NULL,  
  description TEXT,  
  ai_models_used TEXT[] DEFAULT ARRAY[]::TEXT[],  
  is_pinned BOOLEAN DEFAULT FALSE,  
  is_archived BOOLEAN DEFAULT FALSE,  
  is_shared BOOLEAN DEFAULT FALSE,  
  share_token VARCHAR(255) UNIQUE,  
  share_expiry TIMESTAMP,  
  created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,  
  updated_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,  
  deleted_at TIMESTAMP, -- Soft delete  
  total_messages INT DEFAULT 0,  
  total_tokens INT DEFAULT 0,  
  estimated_cost DECIMAL(10,4) DEFAULT 0,  
  INDEX(user_id),  
  INDEX(is_pinned),  
  INDEX(created_at)  
);
```

Messages Table

sql

```
CREATE TABLE messages (  
  id UUID PRIMARY KEY DEFAULT gen_random_uuid(),  
  chat_id UUID NOT NULL REFERENCES chats(id) ON DELETE CASCADE,  
  user_id UUID NOT NULL REFERENCES users(id),  
  role ENUM('user', 'assistant', 'system') NOT NULL,  
  content TEXT NOT NULL,  
  html_content TEXT,  
  ai_model VARCHAR(100),  
  tokens_used INT DEFAULT 0,  
  temperature DECIMAL(3,2) DEFAULT 0.7,  
  max_tokens INT DEFAULT 1000,  
  response_time_ms INT,  
  created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,  
  updated_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,  
  deleted_at TIMESTAMP,  
  INDEX(chat_id),  
  INDEX(user_id),  
  INDEX(created_at)  
);
```

Files Table

sql

```

CREATE TABLE files (
  id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  user_id UUID NOT NULL REFERENCES users(id) ON DELETE CASCADE,
  chat_id UUID REFERENCES chats(id),
  file_name VARCHAR(500) NOT NULL,
  file_type ENUM('pdf', 'image', 'audio') NOT NULL,
  file_size BIGINT, -- in bytes
  s3_url VARCHAR(500) NOT NULL,
  cloudinary_url VARCHAR(500),
  mime_type VARCHAR(100),
  -- For PDFs
  page_count INT,
  extracted_text TEXT,
  faiss_index_id VARCHAR(255),
  text_chunks INT DEFAULT 0,
  -- For Images
  width INT,
  height INT,
  extracted_ocr_text TEXT,
  -- For Audio
  duration INT, -- in seconds
  transcribed_text TEXT,
  language VARCHAR(50),
  created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
  deleted_at TIMESTAMP,
  INDEX(user_id),
  INDEX(file_type)
);

```

Subscriptions Table

```
sql
```

```
CREATE TABLE subscriptions (  
  id UUID PRIMARY KEY DEFAULT gen_random_uuid(),  
  user_id UUID NOT NULL UNIQUE REFERENCES users(id) ON DELETE CASCADE,  
  plan ENUM('free', 'pro', 'enterprise') DEFAULT 'free',  
  stripe_customer_id VARCHAR(255) UNIQUE,  
  stripe_subscription_id VARCHAR(255) UNIQUE,  
  status ENUM('active', 'canceled', 'past_due', 'inactive') DEFAULT 'active',  
  start_date TIMESTAMP DEFAULT CURRENT_TIMESTAMP,  
  renewal_date TIMESTAMP,  
  expiry_date TIMESTAMP,  
  monthly_message_limit INT, -- -1 for unlimited  
  auto_renew BOOLEAN DEFAULT TRUE,  
  created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,  
  updated_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,  
  INDEX(user_id),  
  INDEX(status)  
);
```

API Logs Table

sql

```
CREATE TABLE api_logs (  
  id UUID PRIMARY KEY DEFAULT gen_random_uuid(),  
  user_id UUID REFERENCES users(id),  
  endpoint VARCHAR(255) NOT NULL,  
  method ENUM('GET', 'POST', 'PUT', 'DELETE') NOT NULL,  
  status_code INT NOT NULL,  
  response_time_ms INT,  
  tokens_used INT DEFAULT 0,  
  api_cost DECIMAL(10,6),  
  error_message TEXT,  
  ip_address VARCHAR(50),  
  created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,  
  INDEX(user_id),  
  INDEX(created_at),  
  INDEX(endpoint)  
);
```

Usage Stats Table

sql

```
CREATE TABLE usage_stats (  
  id UUID PRIMARY KEY DEFAULT gen_random_uuid(),  
  user_id UUID NOT NULL REFERENCES users(id) ON DELETE CASCADE,  
  date DATE NOT NULL,  
  messages_sent INT DEFAULT 0,  
  pdfs_analyzed INT DEFAULT 0,  
  images_processed INT DEFAULT 0,  
  tokens_used INT DEFAULT 0,  
  audio_minutes INT DEFAULT 0,  
  total_cost DECIMAL(10,4) DEFAULT 0,  
  created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,  
  UNIQUE(user_id, date),  
  INDEX(user_id),  
  INDEX(date)  
);
```

6.2 Indexes for Performance

sql

```
-- Chat queries
CREATE INDEX idx_chats_user_created ON chats(user_id, created_at DESC);
CREATE INDEX idx_chats_pinned ON chats(user_id, is_pinned, created_at DESC);

-- Message queries
CREATE INDEX idx_messages_chat_time ON messages(chat_id, created_at DESC);
CREATE INDEX idx_messages_search ON messages USING GIN(to_tsvector('english',

-- File queries
CREATE INDEX idx_files_user_type ON files(user_id, file_type, created_at DESC)

-- Usage queries
CREATE INDEX idx_usage_stats_date ON usage_stats(user_id, date DESC);
```

7. API Specifications

7.1 Authentication Endpoints

POST /api/auth/register

json

Request:

```
{
  "email": "user@example.com",
  "username": "johndoe",
  "password": "SecurePass123!",
  "full_name": "John Doe"
}
```

Response (201):

```
{
  "user": {
    "id": "uuid",
    "email": "user@example.com",
    "username": "johndoe",
    "full_name": "John Doe"
  },
  "message": "Verification email sent"
}
```

Errors:

- 400: Invalid email/password format
- 409: Email already exists
- 422: Validation error

POST /api/auth/login

json

Request:

```
{  
  "email": "user@example.com",  
  "password": "SecurePass123!"  
}
```

Response (200):

```
{  
  "access_token": "eyJhbGc...",  
  "refresh_token": "eyJhbGc...",  
  "token_type": "bearer",  
  "expires_in": 900,  
  "user": {  
    "id": "uuid",  
    "email": "user@example.com",  
    "role": "user"  
  }  
}
```

Errors:

- 401: Invalid credentials
- 403: Email not verified

POST /api/auth/refresh

json

Request:

```
{  
  "refresh_token": "eyJhbGc..."  
}
```

Response (200):

```
{  
  "access_token": "eyJhbGc...",  
  "expires_in": 900,  
  "token_type": "bearer"  
}
```

Errors:

- 401: Invalid/expired refresh token

7.2 Chat Endpoints

POST /api/chats/{chat_id}/messages

json

Request:

```
{
  "content": "Explain quantum computing",
  "ai_model": "gpt-4",
  "temperature": 0.7,
  "max_tokens": 2000,
  "attachments": ["uuid-pdf-1", "uuid-image-1"]
}
```

Response (200 - Server-Sent Events / WebSocket):

```
{
  "message_id": "uuid",
  "content": "Quantum computing is...",
  "role": "assistant",
  "tokens_used": 145,
  "ai_model": "gpt-4",
  "response_time_ms": 2341,
  "created_at": "2024-01-15T10:30:00Z",
  "stream": true // Indicates streaming response
}
```

Errors:

- 401: Unauthorized
- 429: Rate limit exceeded
- 503: AI service unavailable

GET /api/chats/{chat_id}/messages?limit=50&offset=0

json

```
Response (200):
{
  "messages": [
    {
      "id": "uuid",
      "chat_id": "uuid",
      "role": "user",
      "content": "Hello",
      "created_at": "2024-01-15T10:00:00Z"
    },
    {
      "id": "uuid",
      "chat_id": "uuid",
      "role": "assistant",
      "content": "Hi! How can I help?",
      "ai_model": "gpt-4",
      "tokens_used": 50,
      "created_at": "2024-01-15T10:01:00Z"
    }
  ],
  "total": 245,
  "limit": 50,
  "offset": 0
}
```

7.3 PDF Chat Endpoints

POST /api/pdf/upload

Request: multipart/form-data

```
{  
  "file": <PDF file>,  
  "chat_id": "uuid (optional)"  
}
```

Response (201):

```
{  
  "file_id": "uuid",  
  "file_name": "document.pdf",  
  "file_size": 2048576,  
  "page_count": 25,  
  "upload_status": "processing",  
  "faiss_index_id": "idx-xyz",  
  "created_at": "2024-01-15T10:00:00Z",  
  "message": "PDF processing started, will be ready in ~30 seconds"  
}
```

Errors:

- 400: Invalid file format
- 413: File too large (>50MB)
- 422: Corrupted PDF

POST /api/pdf/chat

json

```
Request:
{
  "pdf_id": "uuid",
  "question": "What is the main conclusion?",
  "ai_model": "gpt-4"
}
```

```
Response (200):
{
  "answer": "The main conclusion is...",
  "sources": [
    {
      "page": 12,
      "snippet": "Relevant text snippet..."
    }
  ],
  "tokens_used": 234,
  "response_time_ms": 1567
}
```

Errors:

- 404: PDF not found
- 400: PDF still processing

7.4 Image Chat Endpoints

POST /api/image/upload

```
Request: multipart/form-data
{
  "file": <Image file>,
  "chat_id": "uuid (optional)"
}
```

```
Response (201):
{
  "file_id": "uuid",
  "file_name": "photo.jpg",
  "file_size": 1024576,
  "width": 1920,
  "height": 1080,
  "ocr_text": "Extracted text from image...",
  "upload_status": "completed",
  "created_at": "2024-01-15T10:00:00Z"
}
```

```
Errors:
- 400: Invalid image format
- 413: File too large (>20MB)
- 415: Unsupported media type
```

POST /api/image/analyze

```
json
```

Request:

```
{  
  "image_id": "uuid"  
}
```

Response (200):

```
{  
  "description": "A sunny beach scene with...",  
  "objects_detected": ["beach", "ocean", "person", "umbrella"],  
  "colors": ["blue", "yellow", "white"],  
  "text_in_image": "Welcome to Hawaii",  
  "sentiment": "positive",  
  "analysis_type": "general"  
}
```

7.5 Voice Chat Endpoints

POST /api/voice/transcribe

Request: multipart/form-data

```
{  
  "audio": <Audio file>,  
  "language": "en"  
}
```

Response (200):

```
{  
  "text": "What is the capital of France?",  
  "language": "en",  
  "confidence": 0.98,  
  "duration_seconds": 5.2  
}
```

Errors:

- 400: Invalid audio format
- 413: Audio file too large

POST /api/voice/synthesize

json

Request:

```
{
  "text": "The capital of France is Paris.",
  "language": "en",
  "speaker_gender": "female",
  "speech_rate": 1.0
}
```

Response (200):

```
{
  "audio_url": "https://s3.url/audio/uuid.mp3",
  "duration_seconds": 3.2,
  "file_size": 51200,
  "format": "mp3"
}
```

7.6 Admin Endpoints

GET /api/admin/dashboard

json

```
Response (200):
{
  "overview": {
    "total_users": 15420,
    "active_users_today": 3421,
    "active_users_week": 8900,
    "premium_users": 2340,
    "new_users_today": 145
  },
  "messages": {
    "total_today": 45230,
    "total_week": 287456,
    "average_per_user": 32
  },
  "ai_models_usage": {
    "gpt-4": 45,
    "gpt-3.5": 25,
    "gemini": 18,
    "claude": 12
  },
  "revenue": {
    "today": 1200.50,
    "week": 8430.25,
    "month": 35200.75,
    "currency": "USD"
  },
  "server_status": {
    "api_response_time": 245,
    "db_connection": "healthy",
    "cache_hit_rate": 87.5,
    "error_rate": 0.8
  }
}
```

```
}  
}
```

8. Security Requirements

8.1 Authentication & Authorization

- ☒ JWT with 15-minute expiry for access token
- ☒ Refresh token with 7-day expiry
- ☒ Password hashing with Bcrypt (salt rounds: 10)
- ☒ OAuth 2.0 for Google & GitHub
- ☒ Email verification before account activation
- ☒ Rate limiting on auth endpoints (5 attempts/minute)
- ☒ Account lockout after 5 failed login attempts (30 min)

8.2 API Security

- ☒ HTTPS/TLS 1.3 encryption
- ☒ CORS whitelist (specific domains only)
- ☒ Rate limiting (50 requests/minute per user)
- ☒ Request validation with Pydantic
- ☒ SQL injection protection (parameterized queries)
- ☒ XSS protection (input sanitization)
- ☒ CSRF token validation
- ☒ API versioning (/api/v1/)

8.3 Data Protection

- ☒ Encryption at rest (AES-256 for sensitive data)
- ☒ Field-level encryption for passwords
- ☒ PII compliance (GDPR, CCPA)

- ☒ Data retention policy (delete after 90 days if requested)
- ☒ Audit logging for sensitive operations
- ☒ Access control lists (ACL) per resource

8.4 File Security

- ☒ File type validation (whitelist approach)
- ☒ File size limits (50MB for PDFs, 20MB for images)
- ☒ Malware scanning (ClamAV for uploads)
- ☒ NSFW content filtering for images
- ☒ Virus scanning via AWS S3 integration
- ☒ Secure file storage with signed URLs

8.5 Third-party API Security

- ☒ API key rotation (monthly)
- ☒ Environment variable management
- ☒ API rate limiting from providers
- ☒ Error message sanitization (no API key leaks)
- ☒ Request/response logging without sensitive data

8.6 Monitoring & Logging

- ☒ All API requests logged
- ☒ Error tracking (Sentry integration)
- ☒ User activity audit log
- ☒ Failed login attempts tracking
- ☒ Sensitive operation alerts
- ☒ Log retention (90 days)

8.7 Infrastructure Security

- ☒ Firewall rules

- ☒ VPC isolation
 - ☒ Database encryption
 - ☒ Redis password protection
 - ☒ SSH key-based access
 - ☒ DDoS protection
-

9. Deployment & DevOps

9.1 Frontend Deployment (Vercel)

Repository: GitHub (main branch)

Build Command: `npm run build`

Output Directory: `dist/`

Node Version: 18.x

Environment Variables:

- `VITE_API_URL=https://api.smartaichatbot.com`
- `VITE_WS_URL=wss://api.smartaichatbot.com`
- `VITE_GOOGLE_CLIENT_ID=xxx`
- `VITE_GITHUB_CLIENT_ID=xxx`

Deployment Triggers:

- Automatic on push to main
- Preview on PR creation

Optimization:

- Image optimization enabled
- Web fonts optimization
- Code splitting enabled
- Bundle analysis

9.2 Backend Deployment (Railway/Render)

Repository: GitHub (main branch)

Build Command: `pip install -r requirements.txt`

Start Command: `uvicorn app.main:app --host 0.0.0.0 --port 8000`

Environment Variables:

- `DATABASE_URL=postgresql://...`
- `REDIS_URL=redis://...`
- `JWT_SECRET=xxx`
- `OPENAI_API_KEY=xxx`
- `GOOGLE_API_KEY=xxx`
- `AWS_ACCESS_KEY=xxx`
- `AWS_SECRET_KEY=xxx`

Health Check:

- Endpoint: `/api/health`
- Interval: 30 seconds
- Timeout: 10 seconds

Auto-scaling:

- Min instances: 2
- Max instances: 10
- CPU threshold: 70%
- Memory threshold: 80%

9.3 Database Deployment (Supabase PostgreSQL)

Tier: Pro Plan (for production)

Region: Closest to users (e.g., us-east-1)

Backups: Daily automatic backups

Retention: 30 days

Connection pooling: Enabled (100 connections)

SSL Mode: Require

Monitoring:

- Query performance logs
- Connection logs
- Error logs
- Storage usage alerts

9.4 Cache Deployment (Upstash Redis)

Plan: Standard (for production)
Region: Same as backend
Eviction Policy: allkeys-lru
Max Memory: 1GB (upgrade as needed)

Monitoring:

- Hit rate monitoring
- Memory usage
- Command latency
- Connection count alerts

TTL Settings:

- Session tokens: 7 days
- Rate limit counters: 1 hour
- Cache data: 24 hours

9.5 CI/CD Pipeline

yaml

```
name: Deploy

on:
  push:
    branches: [main]
  pull_request:
    branches: [main]

jobs:
  test:
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v3

      # Frontend tests
      - name: Setup Node
        uses: actions/setup-node@v3
        with:
          node-version: '18'

      - name: Install dependencies
        run: npm install

      - name: Run linter
        run: npm run lint

      - name: Run tests
        run: npm run test

      - name: Build
        run: npm run build
```

```
# Backend tests
- name: Setup Python
  uses: actions/setup-python@v4
  with:
    python-version: '3.11'

- name: Install dependencies
  run: pip install -r requirements.txt

- name: Run linter (pylint)
  run: pylint app/

- name: Run tests
  run: pytest

- name: Run integration tests
  run: pytest tests/integration/

deploy:
  needs: test
  runs-on: ubuntu-latest
  if: github.ref == 'refs/heads/main'
  steps:
    - name: Deploy frontend to Vercel
      uses: vercel/action@master
      with:
        vercel-token: ${ secrets.VERCEL_TOKEN }

    - name: Deploy backend to Railway
      run: |
        railway up --service backend
```

9.6 Monitoring & Observability

Tools:

- Error Tracking: Sentry
- Performance Monitoring: New Relic
- Log Aggregation: ELK Stack / LogRocket
- Uptime Monitoring: UptimeRobot
- Analytics: Google Analytics / Mixpanel

Alerts:

- Error rate > 5%
- API response time > 2 seconds
- Database connection issues
- Low disk space
- High CPU usage
- Memory leaks

10. Development Roadmap

Phase 1: MVP (4 weeks)

Weeks 1-2: Project Setup & Auth

- ☐ Repository setup (GitHub)
- ☐ Frontend scaffold (React + Vite)
- ☐ Backend scaffold (FastAPI)
- ☐ Database schema
- ☐ User registration
- ☐ User login with JWT
- ☐ Email verification

Weeks 3-4: Core Chat Features

- ☐ Chat creation/listing
- ☐ Message sending/receiving
- ☐ Basic AI integration (OpenAI only)
- ☐ Chat history storage
- ☐ UI for chat interface
- ☐ Real-time message updates (polling initially)

Deliverable: Basic chat app with one AI model

Phase 2: Enhanced Features (4 weeks)

Weeks 1-2: File Handling

- ☐ PDF upload functionality
- ☐ PDF text extraction
- ☐ Vector database setup (FAISS)
- ☐ Semantic search
- ☐ PDF Q&A feature

Weeks 3-4: Additional AI Features

- ☐ Image upload & analysis
- ☐ OCR integration
- ☐ Code generation feature
- ☐ Grammar fixer
- ☐ Translator feature

Deliverable: Full-featured app with PDF and image support

Phase 3: Advanced Features (3 weeks)

Weeks 1-2: Voice & Real-time

- ☐ Voice recording in browser
- ☐ Speech-to-text integration
- ☐ Text-to-speech integration
- ☐ WebSocket implementation
- ☐ Real-time message streaming

Week 3: Multiple AI Models

- ☐ Google Gemini integration
- ☐ Anthropic Claude integration
- ☐ Groq API integration
- ☐ Ollama self-hosted option
- ☐ Model switching in UI

Deliverable: Voice-enabled app with multiple AI models

Phase 4: Monetization & Scale (4 weeks)

Weeks 1-2: Admin Dashboard

- ☐ User management
- ☐ Analytics dashboard
- ☐ API logs & cost tracking
- ☐ Server monitoring

Weeks 3-4: Subscription System

- ☐ Stripe integration
- ☐ Pricing plans (Free/Pro/Enterprise)
- ☐ Usage tracking & limits
- ☐ Billing page
- ☐ Invoice generation

11. Acceptance Criteria

Feature Completion Checklist

Authentication

- ☐ User can register with email/password
- ☐ User can login with credentials
- ☐ User can login with Google OAuth
- ☐ User can login with GitHub OAuth
- ☐ JWT tokens are refreshed properly
- ☐ Passwords are hashed with bcrypt
- ☐ Email verification works
- ☐ Forgot password flow works

Chat Features

- ☐ User can create new chat
- ☐ User can send text messages
- ☐ User can see message history
- ☐ User can delete messages
- ☐ User can pin important chats
- ☐ User can export chat as JSON/PDF/MD
- ☐ User can share chat via link
- ☐ Real-time message updates work

AI Features

- ☐ Text generation works
- ☐ Code generation works (Python, JavaScript, etc.)

- ☐ Translation works (50+ languages)
- ☐ Grammar fixing works
- ☐ Quiz generation works
- ☐ Model selection works
- ☐ Temperature & token adjustment works

PDF Features

- ☐ User can upload PDF
- ☐ PDF text extraction works
- ☐ Chunking creates appropriate segments
- ☐ Embeddings are generated
- ☐ Q&A on PDF works
- ☐ Multiple PDFs in single chat
- ☐ OCR works on scanned PDFs

Image Features

- ☐ User can upload image
- ☐ Image analysis works
- ☐ OCR text extraction works
- ☐ Q&A on images works
- ☐ Multiple image formats supported

Voice Features

- ☐ User can record audio
- ☐ Audio transcription works
- ☐ Text-to-speech works
- ☐ Multiple language support

Admin Dashboard

- ☐ User statistics visible
- ☐ Revenue tracking works
- ☐ API logs accessible
- ☐ Server health monitored
- ☐ Analytics charts display correctly

Performance

- ☐ API response time < 2 seconds
- ☐ Chat message stream works smoothly
- ☐ File upload < 30 seconds
- ☐ Frontend bundle size < 500KB
- ☐ Lighthouse score > 90

Security

- ☐ HTTPS enforced
- ☐ CORS properly configured
- ☐ Rate limiting active
- ☐ SQL injection protected
- ☐ XSS protection enabled
- ☐ API keys not exposed in logs
- ☐ Files scanned for malware

Testing

- ☐ Unit test coverage > 80%
 - ☐ Integration tests pass
 - ☐ E2E tests cover main flows
 - ☐ Load testing passed (1000 concurrent users)
 - ☐ Security audit passed
-

12. Success Metrics (KPIs)

User Metrics

- Total signups (Target: 10K in first 3 months)
- Daily Active Users (Target: 30% of signups)
- User retention rate (Target: 40% after 30 days)
- Conversion to Pro (Target: 5-10%)

Product Metrics

- Average messages per user per day (Target: 20+)
- Feature adoption rate (Target: >60% use PDFs)
- Chat export/share (Target: >30% use this)
- Average session duration (Target: >15 minutes)

Business Metrics

- MRR (Monthly Recurring Revenue) (Target: \$50K by 6 months)
- CAC (Customer Acquisition Cost) (Target: <\$10)
- LTV (Lifetime Value) (Target: >\$500 per Pro user)
- Churn rate (Target: <5% monthly)

Technical Metrics

- API uptime (Target: 99.9%)
 - Average response time (Target: <500ms)
 - Error rate (Target: <1%)
 - Database query performance (Target: <100ms p95)
-

13. Risk Mitigation

Risk	Impact	Probability	Mitigation
AI API downtime	High	Medium	Fallback models, queuing system
High API costs	High	High	Usage limits, cost monitoring alerts
Data breach	Critical	Low	Encryption, security audits, bug bounty
Database failure	High	Low	Daily backups, multi-region failover
User churn	High	Medium	Feature improvements, engagement emails
Low conversion	Medium	Medium	A/B testing, pricing optimization
Scaling issues	High	Low	Load testing, auto-scaling setup

14. Glossary

- **JWT:** JSON Web Token - For authentication
- **FAISS:** Facebook AI Similarity Search - Vector database
- **Embeddings:** Vector representation of text for semantic search
- **Chunking:** Breaking large texts into manageable pieces
- **Streaming:** Real-time response from AI (token by token)
- **OCR:** Optical Character Recognition - Extract text from images
- **WebSocket:** Protocol for real-time bidirectional communication
- **SaaS:** Software as a Service - Subscription-based software

15. Appendix

A. Technology Comparison Matrix

Aspect	Chosen	Alternative	Why Chosen
Frontend	React	Vue, Svelte	Larger ecosystem, easier hiring
Backend	FastAPI	Django, Flask	Async support, modern, fast
LLM	Multiple	Single	Flexibility, cost optimization
Vector DB	FAISS	Pinecone, Weaviate	Self-hosted, cost-effective
Database	PostgreSQL	MongoDB	Structured data, ACID compliance
Cache	Redis	Memcached	Rich data structures, pub/sub
Storage	AWS S3	Google Cloud	Cost-effective, reliable

B. API Rate Limits

Free Tier:

- 50 messages/day
- 2 PDF uploads/day
- 5 images/day
- 2 voice interactions/day

Pro Tier:

- Unlimited messages
- Unlimited uploads
- 1000 API calls/day
- Priority support

Enterprise:

- Custom limits
- Dedicated support
- SLA guarantees

C. Error Codes

200 - OK
201 - Created
400 - Bad Request
401 - Unauthorized
403 - Forbidden
404 - Not Found
429 - Rate Limit Exceeded
500 - Internal Server Error
503 - Service Unavailable

Document End

Prepared by: Product & Engineering Team

Approval Status: Pending

Next Review Date: 30 days from approval